

SQL feat PHP

PDO

Dans ce module, je vous présente les grandes lignes de PDO mais pour mieux comprendre je vous invite à cloner le repository ci-dessous :

https://github.com/ApicPlateforme/pdo_example.git

Vous y trouverez une architecture propre d'un projet HTML/CSS/PHP avec PDO (pas de JS) et pas de programmation orientée objet.

Qu'est ce que PDO ?

PDO (PHP Data Objects) est une interface orientée objet pour accéder à des bases de données depuis PHP.

Elle permet :



- d'interagir avec plusieurs SGBD (MySQL, PostgreSQL, SQLite, etc.),
- d'utiliser des **requêtes préparées** (contre les injections SQL),
- une écriture plus propre et maintenable.

Connexion à la base de données

```
<?php
$dsn =
'mysql:host=localhost;dbname=ma_base;charset=utf8';
$user = 'root';
$password = '';

try {
    $pdo = new PDO($dsn, $user, $password);

    // Option essentielle : affiche les erreurs sous forme
    d'exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
```



```
} catch (PDOException $e) {  
    // Ne JAMAIS afficher ça en production  
    echo "Erreur de connexion : " . $e->getMessage();  
}
```

À retenir :

- `new PDO(...)` crée une **connexion à la base**.
- Le **DSN** décrit le type de base (`mysql`), l'hôte (`localhost`), la base (`ma_base`) et l'encodage (`utf8`).
- `setAttribute(...ERRMODE_EXCEPTION)` : très important pour **gérer proprement les erreurs**.
- Toujours entourer d'un `try/catch`

ATTENTION ! Il ne faut jamais écrire en dur les informations de connexion comme ci-dessus (cela vaut pour toutes informations sensibles). A la place nous utiliserons un `.env` (ou `.env.local`) comme dans le repository d'exemple.



Utilisation de PDO dans une fonction

Pour utiliser PDO dans une fonction il suffit de placer votre objet PDO (en principe, \$pdo) dans le paramètre de celle-ci :

```
function findUserById(PDO $pdo, int $id): ?array {  
    $stmt = $pdo->prepare("SELECT * FROM utilisateurs WHERE  
id = :id");  
    $stmt->execute(['id' => $id]);  
    return $stmt->fetch(PDO::FETCH_ASSOC) ?: null;  
}
```

Appel de la fonction :

```
$user = findUserById($pdo, 3);
```



Utilisation de requêtes préparées

Une **requête préparée** est un mécanisme sécurisé permettant de **séparer la requête SQL de ses valeurs**. Cela empêche toute tentative d'injection SQL.

Mauvais exemple (à ne jamais faire) :

```
// Ne faites jamais ça !
$email = $_POST['email'];
$sql = "SELECT * FROM utilisateurs WHERE email = '$email'";
$result = $pdo->query($sql);
```

Ici, un pirate pourrait injecter du SQL (ex: ' OR 1=1 --) via le champ email.

Bon exemple : requête préparée avec `prepare()` et `execute()`

```
function findUserByEmail(PDO $pdo, string $email): ?array {
    $sql = "SELECT * FROM utilisateurs WHERE email = :email";
    $stmt = $pdo->prepare($sql);
    $stmt->execute(['email' => $email]);
```



```
return $stmt->fetch(PDO::FETCH_ASSOC) ?: null;  
}
```

Appel de la fonction :

```
$user = findUserByEmail($pdo, 'test@example.com');
```

Ici nous avons une chaîne de caractères mais si vous insérez une variable (\$email) qui provient d'un formulaire il faut ABSOLUMENT valider le contenu avant de le mettre en base de données !!!



Exercice

Créer une page formulaire d'inscription avec les champs suivants :

- email
- password
- confirm_password

Créer une page formulaire de connexion avec les champs :

- email
- password

Vous devez enregistrer dans une BDD que vous aurez créé au préalable les utilisateurs qui s'inscrivent. L'email doit être unique et le mot de passe ne doit pas être enregistré en clair ! L'inscription n'est valable que si les champs "password" et "confirm_password" sont identiques et que la longueur du password est supérieure à 8. Si l'inscription n'est pas valide un message d'erreur doit être affiché.

La page de connexion doit être fonctionnelle c'est-à-dire comparer les identifiants saisis avec les identifiants enregistrées. Si les identifiants sont valides, l'utilisateur est



redirigé vers une page dashboard.php sinon un message d'erreur est indiqué sur la page de login.

La page dashboard ne doit pas être accessible si l'utilisateur n'est pas connecté (redirection vers la page de login). Elle doit posséder un bouton "Déconnexion".

Bonus : Affiche la liste des utilisateurs inscrits dans la page dashboard.php